SELLER & INVENTORY MANAGEMENT SYSTEM

Database Management System – Documentation

1. Introduction

The Seller & Inventory Management System is a relational database designed to manage seller details, product assignments, and inventory stock. It supports seller operations, stock monitoring, updates, and inventory reports.

2. Objectives

Store and manage seller information.

Assign products to sellers and track supplied products.

Maintain stock quantity and stock status.

Identify available, low-stock, and out-of-stock products.

Generate seller-wise and inventory analysis reports.

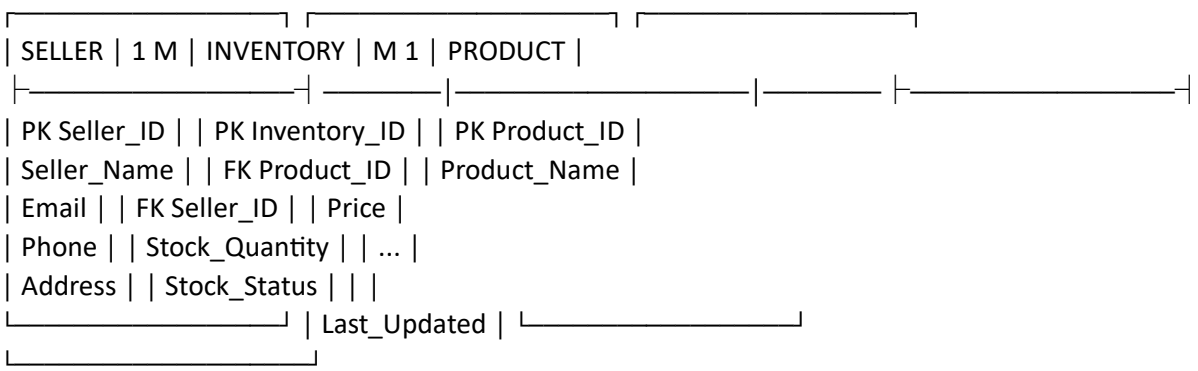
3. Seller & Inventory Table Design

Table	Column	Data Type	Constraints
Seller	Seller_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Seller	Seller_Name	VARCHAR(100)	NOT NULL
Seller	Email	VARCHAR(100)	NOT NULL, UNIQUE
Seller	Phone	VARCHAR(15)	NOT NULL, UNIQUE
Seller	Address	VARCHAR(200)	NOT NULL
Inventory	Inventory_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Inventory	Product_ID	INT	NOT NULL, FOREIGN KEY
Inventory	Seller_ID	INT	NOT NULL, FOREIGN KEY
Inventory	Stock_Quantity	INT	NOT NULL, CHECK >= 0
Inventory	Stock_Status	VARCHAR(20)	NOT NULL, CHECK
Inventory	Last_Updated	DATE	NOT NULL

Note: The Product table is referenced by Product_ID. In a complete database, Product(Product_ID) should be created before Inventory so the foreign-key relationship can be established.

4. Updated ER Diagram

Conceptual relationship: Seller (1) ──< Inventory >── (1) Product. A seller can supply multiple products, and a product can appear in inventory records. Inventory acts as the relationship/stock table.



5. Sample Seller Data

Seller_ID	Seller_Name	Email	Phone	Address
1	Arun Traders	arun@traders.com	9876500001	Chennai
2	Bala Electronics	bala@electronics.com	9876500002	Coimbatore
3	City Mart	citymart@gmail.com	9876500003	Madurai
4	Digi World	digi@world.com	9876500004	Salem
5	Green Stores	green@stores.com	9876500005	Trichy

6. Sample Inventory Data

Inventory_ID	Product_ID	Seller_ID	Stock_Quantity	Stock_Status	Last_Updated
1	101	1	25	Available	2026-09-25
2	102	1	7	Low Stock	2026-09-25
3	103	2	0	Out of Stock	2026-09-25
4	104	2	42	Available	2026-09-25
5	105	3	15	Available	2026-09-25
6	106	3	4	Low Stock	2026-09-25
7	107	4	60	Available	2026-09-25
8	108	4	0	Out of Stock	2026-09-25
9	109	5	9	Low Stock	2026-09-25
10	110	5	30	Available	2026-09-25

7. Seller & Product Operations

Operation	SQL Query

Add seller details	INSERT INTO Seller (Seller_Name, Email, Phone, Address) VALUES ('New Seller','new@seller.com','9876511111','Chennai');
Assign product to seller	INSERT INTO Inventory (Product_ID, Seller_ID, Stock_Quantity, Stock_Status, Last_Updated) VALUES (111, 1, 20, 'Available', CURDATE());
Display products supplied by each seller	SELECT s.Seller_Name, i.Product_ID, i.Stock_Quantity FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID ORDER BY s.Seller_Name;
Count products supplied by each seller	SELECT s.Seller_Name, COUNT(DISTINCT i.Product_ID) AS Product_Count FROM Seller s LEFT JOIN Inventory i ON s.Seller_ID=i.Seller_ID GROUP BY s.Seller_ID, s.Seller_Name;
Update seller details	UPDATE Seller SET Phone='9876599999', Address='Chennai' WHERE Seller_ID=1;

8. Inventory Operations

Operation	**SQL Query**
Display available products	SELECT * FROM Inventory WHERE Stock_Quantity > 0;
Find out-of-stock products	SELECT * FROM Inventory WHERE Stock_Quantity = 0;
Find stock less than 10	SELECT * FROM Inventory WHERE Stock_Quantity < 10;
Update stock quantity	UPDATE Inventory SET Stock_Quantity=35, Stock_Status='Available', Last_Updated=CURDATE() WHERE Inventory_ID=2;
Delete discontinued inventory	DELETE FROM Inventory WHERE Inventory_ID=8;

9. Inventory Analysis Report

Report	**SQL Query**
Seller-wise product report	SELECT s.Seller_Name, COUNT(i.Product_ID) AS Total_Products FROM Seller s LEFT JOIN Inventory i ON s.Seller_ID=i.Seller_ID GROUP BY s.Seller_ID, s.Seller_Name;
Stock availability report	SELECT Stock_Status, COUNT(*) AS Product_Count FROM Inventory GROUP BY Stock_Status;

Total available products	SELECT COUNT(*) AS Available_Product_Records FROM Inventory WHERE Stock_Quantity > 0;
Out-of-stock products	SELECT COUNT(*) AS Out_of_Stock_Records FROM Inventory WHERE Stock_Quantity = 0;
Highest stocked product	SELECT * FROM Inventory WHERE Stock_Quantity=(SELECT MAX(Stock_Quantity) FROM Inventory);
Average inventory quantity	SELECT AVG(Stock_Quantity) AS Average_Inventory_Quantity FROM Inventory;

10. Expected Analysis from Sample Data

Using the sample inventory data before the delete operation: there are 8 inventory records with stock, 2 out-of-stock records, and 10 total inventory records. The highest stock quantity is 60 units. The average stock quantity is 19.2 units. Records with stock below 10 are Inventory_ID 2, 6 and 9.

11. SQL Script

```
CREATE DATABASE SellerInventoryDB;
USE SellerInventoryDB;

CREATE TABLE Seller (
Seller_ID INT PRIMARY KEY AUTO_INCREMENT,
Seller_Name VARCHAR(100) NOT NULL,
Email VARCHAR(100) NOT NULL UNIQUE,
Phone VARCHAR(15) NOT NULL UNIQUE,
Address VARCHAR(200) NOT NULL
);

-- Product table must already exist with Product_ID as PRIMARY KEY.
CREATE TABLE Inventory (
Inventory_ID INT PRIMARY KEY AUTO_INCREMENT,
Product_ID INT NOT NULL,
Seller_ID INT NOT NULL,
Stock_Quantity INT NOT NULL CHECK (Stock_Quantity >= 0),
Stock_Status VARCHAR(20) NOT NULL CHECK (Stock_Status IN ('Available','Low Stock','Out of Stock')),
Last_Updated DATE NOT NULL,
FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID),
FOREIGN KEY (Seller_ID) REFERENCES Seller(Seller_ID)
);

INSERT INTO Seller (Seller_Name, Email, Phone, Address) VALUES
('Arun Traders','arun@traders.com','9876500001','Chennai'),
('Bala Electronics','bala@electronics.com','9876500002','Coimbatore'),
('City Mart','citymart@gmail.com','9876500003','Madurai'),
('Digi World','digi@world.com','9876500004','Salem'),
('Green Stores','green@stores.com','9876500005','Trichy');
```

```
INSERT INTO Inventory
(Product_ID, Seller_ID, Stock_Quantity, Stock_Status, Last_Updated) VALUES
(101,1,25,'Available','2026-09-25'),
(102,1,7,'Low Stock','2026-09-25'),
(103,2,0,'Out of Stock','2026-09-25'),
(104,2,42,'Available','2026-09-25'),
(105,3,15,'Available','2026-09-25'),
(106,3,4,'Low Stock','2026-09-25'),
(107,4,60,'Available','2026-09-25'),
(108,4,0,'Out of Stock','2026-09-25'),
(109,5,9,'Low Stock','2026-09-25'),
(110,5,30,'Available','2026-09-25');

-- CRUD / Seller operations
INSERT INTO Seller (Seller_Name, Email, Phone, Address)
VALUES ('New Seller','new@seller.com','9876511111','Chennai');

UPDATE Seller SET Phone='9876599999', Address='Chennai' WHERE Seller_ID=1;

SELECT s.Seller_Name, i.Product_ID, i.Stock_Quantity
FROM Seller s JOIN Inventory i ON s.Seller_ID=i.Seller_ID
ORDER BY s.Seller_Name;

SELECT s.Seller_Name, COUNT(DISTINCT i.Product_ID) AS Product_Count
FROM Seller s LEFT JOIN Inventory i ON s.Seller_ID=i.Seller_ID
GROUP BY s.Seller_ID, s.Seller_Name;

-- Inventory operations
SELECT * FROM Inventory WHERE Stock_Quantity > 0;
SELECT * FROM Inventory WHERE Stock_Quantity = 0;
SELECT * FROM Inventory WHERE Stock_Quantity < 10;

UPDATE Inventory
SET Stock_Quantity=35, Stock_Status='Available', Last_Updated=CURDATE()
WHERE Inventory_ID=2;

DELETE FROM Inventory WHERE Inventory_ID=8;

-- Reports
SELECT s.Seller_Name, COUNT(i.Product_ID) AS Total_Products
FROM Seller s LEFT JOIN Inventory i ON s.Seller_ID=i.Seller_ID
GROUP BY s.Seller_ID, s.Seller_Name;

SELECT Stock_Status, COUNT(*) AS Product_Count
FROM Inventory GROUP BY Stock_Status;

SELECT COUNT(*) AS Available_Product_Records
FROM Inventory WHERE Stock_Quantity > 0;
```

```
SELECT COUNT(*) AS Out_of_Stock_Records
FROM Inventory WHERE Stock_Quantity = 0;

SELECT * FROM Inventory
WHERE Stock_Quantity=(SELECT MAX(Stock_Quantity) FROM Inventory);

SELECT AVG(Stock_Quantity) AS Average_Inventory_Quantity
FROM Inventory;
```

12. Conclusion

The Seller & Inventory Management System provides a structured way to manage sellers, product assignments, and inventory stock. Primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints help maintain data integrity. The reporting queries support stock monitoring and seller-wise analysis.